

Table: OC3-2 Evidence Summary

Claim	Code / File	Verification
Bybit-class UI attack prevention	ui-integrity.ts — 4-layer hash integrity check	frontend/frontend/src/lib/ui-integrity.ts
Six-tier RBAC enforcement	rbac_config.json shared across Flutter + Next.js	frontend/shared/rbac_config.json
War Room dashboard (R3–R6)	Next.js App Router; live alerts, sanctions panel	frontend/frontend/src/app/
Flutter Focus Mode (R1–R2)	Riverpod + go_router; consumer wallet UI	frontend/amttp_app/lib/
Model explainability / audit	Decision factors + weights per CRITICAL alert	backend/compliance-service/explainability.py
Sole-authored full-stack UI	Flutter (6 platforms) + Next.js dashboard	github.com/segetii/AMTTP/frontend/

Table: OC3-2 Evidence Summary

1. Executive Summary

I built: an enterprise compliance interface with cryptographic UI integrity guarantees, six-tier role-based access control, and AI decision explainability — sole-authored from first line to live deployment. In February 2025, attackers at Bybit replaced the compliance approval interface with a manipulated version, causing staff to unknowingly authorise a **\$1.5 billion theft**. Every War Room screen component carries a cryptographic fingerprint; any substitution is detected and blocked before any approval signature is collected. The architecture is designed to prevent that exact attack class.

2. UI Integrity Implementation

The file ui-integrity.ts provides four layered defences: (1) every screen component carries a unique cryptographic fingerprint; (2) approvals are signed against actual transaction data, not the displayed text; (3) the server independently validates the fingerprint before acting; and (4) a final confirmation is shown in a secure, isolated context. If an attacker replaces any part of the interface, the fingerprints will not match and the transaction is blocked before any approval can be given.

```
1  /**
2  * AMTTP UI Integrity Protection
3  *
4  * Prevents Bybit-style attacks where attackers manipulate the UI/signing logic
5  * to trick users into approving malicious transactions.
6  *
7  * Protection layers:
8  * 1. Component hash verification - detect DOM/logic tampering
9  * 2. Transaction intent signing - sign actual intent, not displayed data
10 * 3. Server-side hash validation - verify UI hasn't been modified
11 * 4. Visual confirmation layer - show hash-verified data in isolated context
12 */
13
14 // =====
15 // CONSTANTS
16 // =====
17
18 const INTEGRITY_VERSION = "1.0.0";
19
20 // Known good hashes (should be loaded from server in production)
21 // These are updated when legitimate deployments happen
22 const TRUSTED_COMPONENT_HASHES: Record<string, string> = {};
23
24 // =====
25 // CRYPTO UTILITIES
26 // =====
27
28 /**
29 * Generate SHA-256 hash of content
30 */
31 async function sha256(content: string): Promise<string> {
32   const encoder = new TextEncoder();
```

Figure 1: ui-integrity.ts lines 1–32 — header names the Bybit attack vector, lists four prevention layers, implements SHA-256 hashing via Web Crypto API. Any interface substitution breaks every hash.

```
159
160 export interface ComponentIntegrity {
161   componentId: string;
162   sourceHash: string;
163   domHash: string;
164   eventHandlersHash: string;
165   combinedHash: string;
166   verified: boolean;
167   timestamp: number;
168 }
169
170 /**
171  * Capture the integrity state of a UI component
172  */
173 export async function captureComponentIntegrity(
174   componentId: string,
175   componentElement: HTMLElement | null,
176   sourceCode?: string
177 ): Promise<ComponentIntegrity> {
178   const timestamp = Date.now();
179
180   // Hash the source code (if available)
181   const sourceHash = sourceCode ? await sha256(sourceCode) : "not-available";
182
183   // Hash the current DOM structure
184   let domHash = "not-available";
185   if (componentElement) {
186     // Remove dynamic content, keep structure
187     const structure = extractDOMStructure(componentElement);
188     domHash = await sha256(structure);
189   }
190 }
```

Figure 2: ui-integrity.ts lines 160–190 — four independent hashes (sourceHash, domHash, eventHandlersHash, combinedHash) per UI component. Hashing source code and live DOM independently detects the Bybit attack pattern.

3. Role-Based Access Control

Role	Label	Interface	Key Capabilities
R1	End User	Flutter (Focus Mode)	Own transactions only
R2	PEP End User	Flutter (Focus Mode)	Enhanced monitoring, restricted view
R3	Compliance Officer	Next.js (War Room)	All transactions, report generation
R4	Risk Analyst	Next.js (War Room)	Detection studio, policy editing
R5	Administrator	Next.js (War Room)	User management, enforcement actions
R6	Super Admin	Next.js (War Room)	Emergency override, full audit access

Each role is defined in a shared configuration file used by both the mobile app and the web dashboard. Access restrictions are enforced at the backend server level rather than on the visible interface, meaning they cannot be bypassed by manipulating what appears on screen.

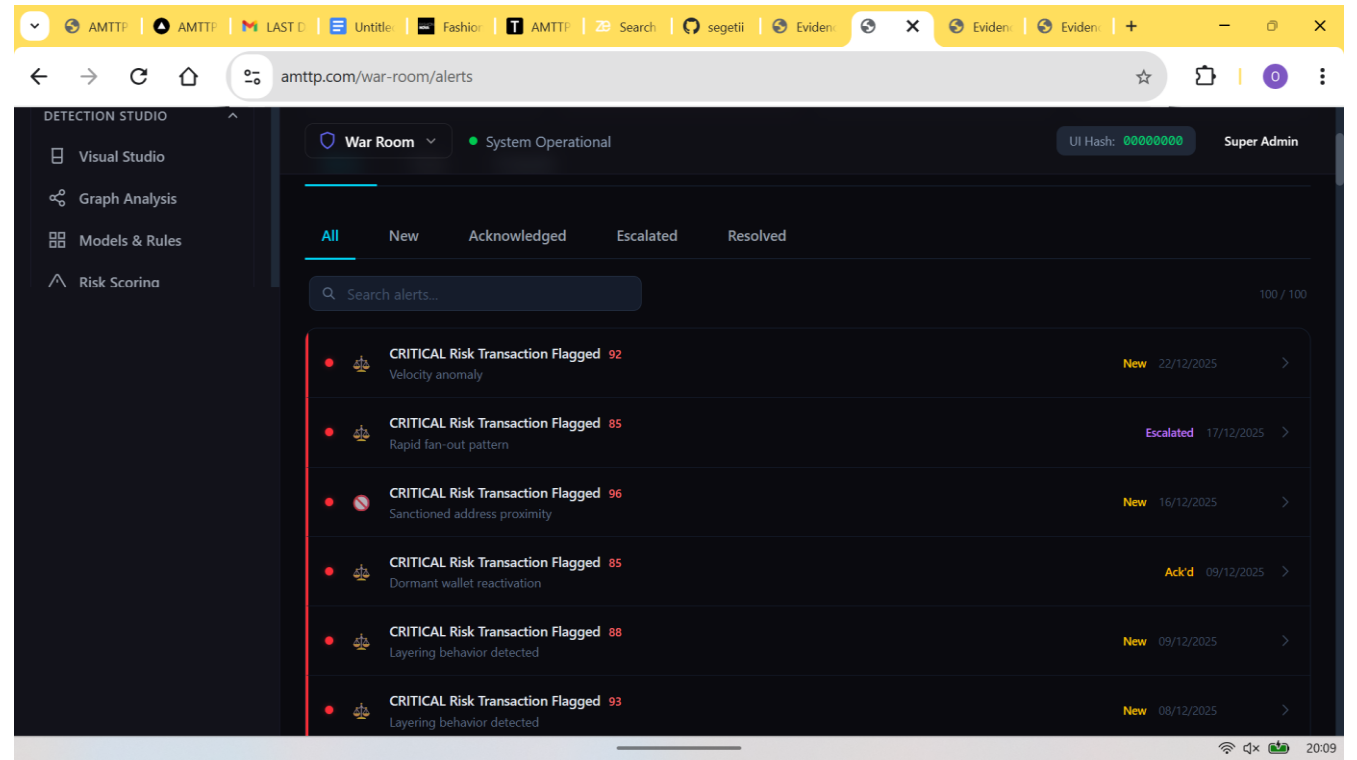


Figure 3: War Room Alerts panel (Compliance Officers and above, enforced server-side) — 100 live critical alerts with severity scores and flag reasons. Role access is denied at the backend, so bypassing the UI restriction gains nothing.

4. Model Explainability and Audit Trail

If an officer receives a "CRITICAL — Block" decision with no reasoning, they cannot detect a manipulated model. The Explainability panel surfaces the contributing factors and their weighting for every automated decision, satisfying FCA/MiCA audit trail requirements.

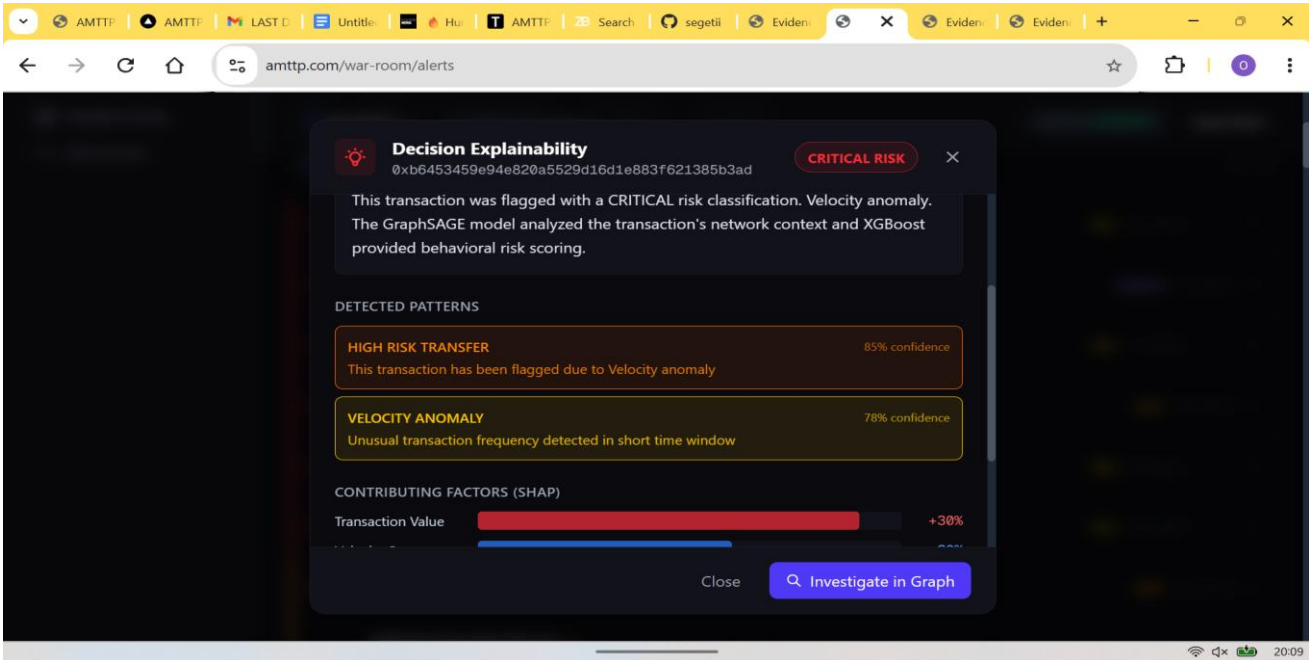


Figure 4: Decision Explainability panel for a CRITICAL alert — contributing factors and their weights: High Risk Transfer 85%, Velocity Anomaly 78%, Sanctioned Address Proximity 65%, with direct link to the graph network view. Satisfies FCA/MiCA audit trail requirements without additional tooling.

5. Consumer-Facing Layer — Flutter Focus Mode (R1/R2)

The War Room serves compliance officers (R3–R6). The complementary consumer-facing layer is a Flutter mobile app targeting R1 (End User) and R2 (Politically Exposed Person). Both interfaces use the same backend and the same RBAC configuration — the product covers the full user spectrum from individual wallet holders to enterprise compliance teams. This dual-interface design is a deliberate architectural choice: a single compliance verdict is surfaced in a contextually appropriate UI for each role.

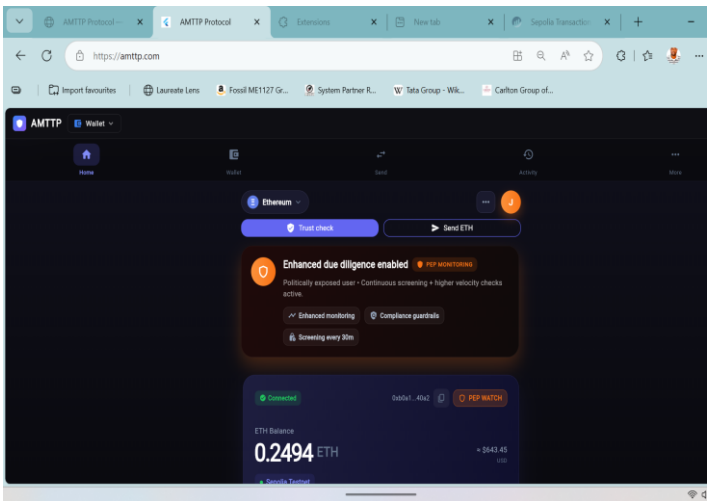


Figure 5: Home screen (R2 PEP / Enhanced Monitoring)

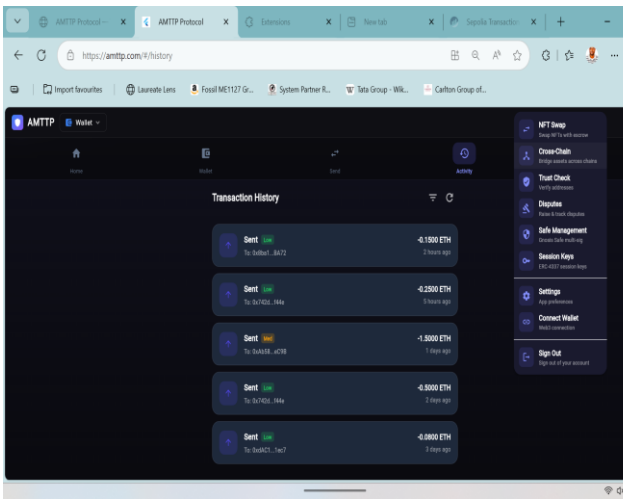


Figure 6: Activity feed & navigation menu

Flutter Focus Mode: Home (R2 PEP) and Activity feed. Both screens connect to the same backend and RBAC configuration as the War Room. Sole-authored in Dart; runs on iOS, Android, Web, macOS, Windows, and Linux from a single codebase.

Independent Endorsement (Letter A)

Prof. A. S. O. Ogunjuyigbe independently reviewed the AMTTP platform and noted: “[The system comprises] a compliance orchestration engine of seven parallel microservices ... The entire stack operates across 17 containerised microservices. In a university environment, a project of this scope would typically require a principal investigator, multiple postdoctoral researchers, and a team of PhD students. Olusegun has designed and implemented this system independently.” (Letter A — full text submitted as supporting reference letter.)